
Deep Learning Challenge : Predicting French electrical load over a year¹

AUTHORS

Félix LAPLANTE
Samuel MOLANO QUINTANA

¹<https://github.com/felixlaplante/dl-project>

1 Relevance

Electricity is a ubiquitous source of energy in the modern world upon which both households and industries rely on a day-to-day basis. Although electrical consumption may exhibit large variations at different time scales (hourly, daily, and throughout the year), production has to precisely match the demand as there currently exist no means of large-scale electricity storage, and any discrepancy might lead to complete blackouts. To avoid these catastrophic scenarios without regulating the demand, it is therefore of utmost importance to be able to predict the electrical load for a given term and a given spatial resolution in order to adapt the production in advance. Obviously, although day-ahead predictions will always be better than month or even year-ahead forecasts, as previous day data may not be available at all times, a model based on calendar and meteorological data still proves very relevant in the field. In this project, we aim at providing the best predictions of the 30-minute electrical load in France at multiple scales over a year, based on previous recordings ranging from 2017 to 2021.

2 Introduction

Trying to predict electrical load over the course of a whole calendar year is a particularly difficult task, as the challenges that arise for daily or weekly prediction tasks are exacerbated by the lack of previous data. We will consider for our task that we have reliable weather data for the whole year we are trying to predict, as well as a robust estimate for the CO₂ emissions of France for that year.

In order to predict electrical load at an urban, regional and national scale, we will use Deep Learning (DL) methods, such as Long Short-Term Memory (LSTM) and Multi-Layered Perceptron (MLP). MLP is the standard modern feedforward neural network, whereas LSTM is a type of Recurrent Neural Network (RNN). LSTMs were first introduced in 1997, as a potential solution to the vanishing/exploding gradient problem commonly encountered by traditional RNNs [1]. As its name suggests, this type of architecture works with long-term and short-term memories at the same time. An LSTM unit is typically composed of a cell and three gates: a forget gate, an input gate and an output gate. The forget gate determines what percentage of the long-term memory will be remembered. The input gate determines how we should update the Long-Term Memory and the output gate determines how we should update the Short-Term Memory. That final Short-Term Memory produced by the output gate is the final output of the unit.

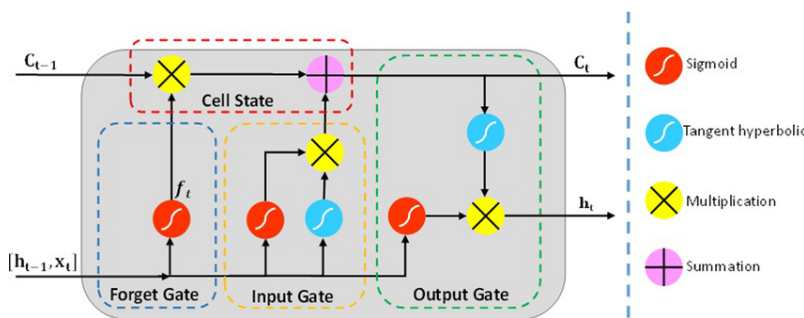


Figure 1: Structure of a LSTM unit [2]

Using Deep Learning methods to predict electrical load has been studied since the early 1990s. The use of Multi-Layered Perceptron models to time-series data is the first natural approach to a problem like this. It has been shown that it can yield better results than other models that are not based on DL techniques [3], although more classical statistical learning methods still prove to be relevant and competitive [4]. Among Deep Learning techniques, RNNs are a natural choice, in that they are made to process sequence data and maintain the information state during the evolution of the sequence. Their use in time-series data prediction tasks has been well studied, even in the case of electricity consumption over a region [5, 6]. In this context, this type of model generally outperforms feed-forward neural networks.

3 Visualizations and data preprocessing

3.1 Datasets

At our disposal, we had access to two datasets:

- the *Météo France* SYNOP meteorological dataset, which logs weather parameters at three-hour intervals in 40 different stations throughout France (parquet format).
- the 30-minute electrical load at multiple geographical scales, ranging from February 2017 to December 2022. The recorded locations include: France, its 13 regions and the 11 largest cities, thereby constituting a dataframe with 25 features (csv format).

Notably, as shown in the appendix (see figure A1), the city load includes many missing values, which is an issue that has to be taken into consideration when training the model. However, the proposed architecture does not require any interpolation. Indeed, large chunks of contiguous data are missing, and any attempt at predicting the missing values would be counterproductive, as it is of similar difficulty to the main project and would entail overfitting.

The SYNOP dataset, also exhibiting numerous missing values, was pruned by throwing away any columns with a mean frequency of missing values larger than 5%.

In addition to these two datasets, we included a third dataset, available at *Our World in Data* ², comprising the annual CO₂ emissions in France, as well as most other countries. Since the values are given yearly, they were set to the first of July, corresponding to the middle of the year.

All datasets were loaded using the *pandas* library, and the date was standardized in the UTC format. At last, French bank holidays were imported using the *holidays* Python library.

3.2 Data interpolation

Since the three main datasets do not share the same temporal resolution, it is necessary to interpolate the weather and CO₂ datasets to 30-minute intervals, in order to match the electrical load. Multiple options were considered, mainly **linear** or **cubic** spline interpolation, as well as **kernel regression**. In

²<https://ourworldindata.org/co2-and-greenhouse-gas-emissions>

our experiments, we downloaded hourly temperature data that was subsampled in order to evaluate the Root Mean Square Error (**RMSE**) of the three different interpolation methods. Although kernel regression proved to be the most precise, its poor scalability compared to the other methods was deemed too time-consuming, and this method was ruled out. Cubic spline interpolation gave similar metrics as kernel regression, with the notable advantage of being highly efficient; at last, linear interpolation was significantly worse.

As a result, the SYNOP and the CO₂ emissions datasets were both interpolated using cubic splines.

3.3 Scaling

Continuous variables, whether they are explanatory or load values, were scaled by standardizing them to enhance the performance of the neural network. Mathematically, for a sample $x_1, \dots, x_n$, the transformed data $\tilde{x}$ is given by:

$$\tilde{x}_i = \frac{x_i - \frac{1}{n} \sum_{i=1}^n x_i}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \frac{1}{n} \sum_{i=1}^n x_i)^2}} = \frac{x_i - \bar{x}}{\hat{\sigma}}.$$

3.4 Visualization of the electrical load

To better grasp the temporal features of the electrical load, we represented on the same graphic the raw France load, as well as a 30-day rolling mean, the latter being much more useful. The effects of the spring 2020 lockdown following the COVID-19 outbreak (see the orange dashed lines) include a notable reduction in the load values. An annual seasonality is observed, with higher values during the winter due to heating, but the time series still exhibits large variations from one year to another, leading to believe more covariables are at play than just calendar ones. We also do notice a substantial drop in the electrical consumption in August, each year, correlated with the summer break.

A plot of the electrical load during the first week of the dataset was also represented, and helps understand the temporality of the load peaks at a daily scale. A substantial drop also occurs during the weekend.

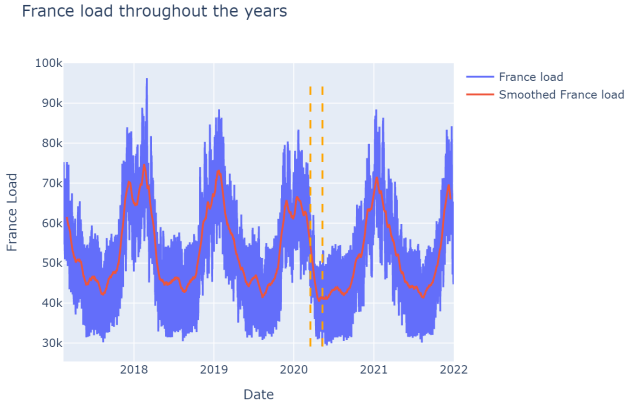


Figure 2: France electrical load from 2017 to 2022

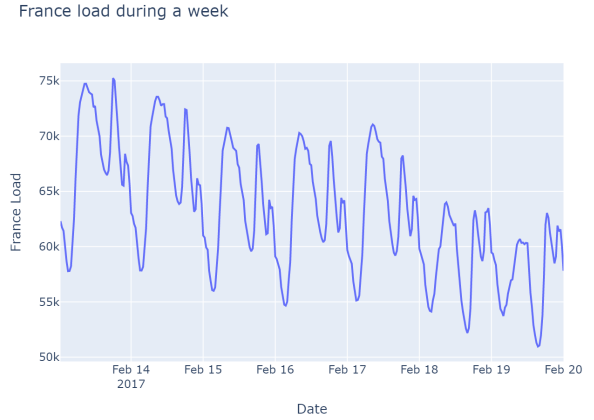


Figure 3: France weekly electrical load

4 Feature engineering and variable selection for load modeling

4.1 Calendar variables

4.1.1 Fourier features and absolute time

To encode the periodic nature of the electrical load at two different time scales (daily and yearly), we present an intuitive method transforming the absolute time into Fourier features. Absolute date can also be kept to model long-term variations, but at the expense of risking poor generalization.

Let $\parallel$ designate the mathematical column-wise concatenation.

Then, let $t_{\text{day}}^{(i)} \in \{1, \dots, T_{\text{day}} = 366\}$ and $t_{\text{mod}}^{(i)} \in \{1, \dots, T_{\text{mod}} = 86400\}$, respectively, the time of year in days since Januray first, and the number of minutes spent since midnight for the i -th sample.

Then, the Fourier features for each time scale up to a frequency f , set to $f = 12$ in practice, are defined as follows:

$$F_{\text{day}}^{(i)} = \parallel_{k=1}^{f_{\text{day}}} \left(\cos\left(\frac{2\pi}{T_{\text{day}}} t_{\text{day}}^{(i)}\right), \sin\left(\frac{2\pi}{T_{\text{day}}} t_{\text{day}}^{(i)}\right) \right) \in \mathbb{R}^{2f_{\text{day}}},$$

$$F_{\text{mod}}^{(i)} = \parallel_{k=1}^{f_{\text{mod}}} \left(\cos\left(\frac{2\pi}{T_{\text{mod}}} t_{\text{mod}}^{(i)}\right), \sin\left(\frac{2\pi}{T_{\text{mod}}} t_{\text{mod}}^{(i)}\right) \right) \in \mathbb{R}^{2f_{\text{mod}}}.$$

4.1.2 Categorical variables

As noted previously and illustrated below, the day of the week has a great impact on the electrical load. Therefore, the day of week was one-hot encoded with seven modalities, and is overparametrized because the class is a vertex of the simplex S_7 whose dimension is 6.

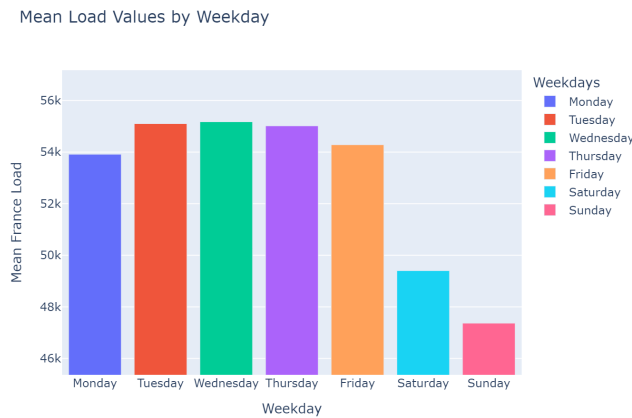


Figure 4: Weekday load

Since the dates are encoded following the UTC convention, to retain the daylight savings information, we also encode them in the same manner as the weekday.

Lastly, French bank holidays are encoded following the exact same methodology as before, because they represent rare events that alter both domestic and industrial activities.

4.2 Meteorological variables

As discussed previously, the SYNOP dataset consists of 40 stations recording metrics like temperature, wind speed, atmospheric pressure, humidity level... To model the situation, we had to select a subsample of the meteorological features we deemed relevant, and then multiple possibilities were considered:

- keep all the weather stations, *as is*, at the expense of the input dimension.
- calculate means by aggregating stations located in the same region.
- employ spatial interpolation techniques such as inverse-square-distance interpolation to estimate the weather parameters at given coordinates.

In our tests, the first performed significantly better, somewhat surprisingly, as the input dimension explodes significantly.

As for the relevant meteorological features, keeping in mind the proposed LSTM architecture, smoothed or tendencies were less useful, because they could be deduced from any sequence. To determine the relevant variables, we trained a Generalized Additive Model (GAM) to study the relationship between the total electrical load in France and potential explanatory variable candidates.

Obviously, as illustrated below, air temperature has the largest impact on electricity consumption, due to heating and air conditioning. An inflection point seems to occur at around 27 degrees.

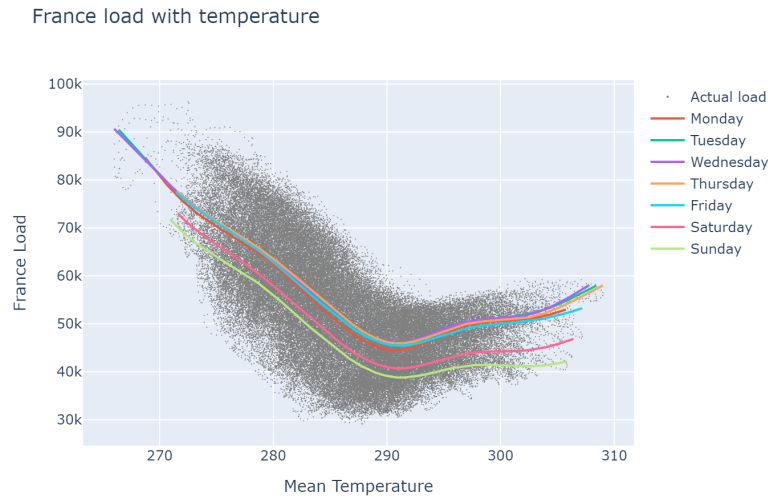


Figure 5: GAM fit of crossed effect between temperature and weekdays

In an attempt at maintaining relative frugality of the model, in addition to **air temperature**, **soil temperature**, **wind speed** and **atmospheric pressure** were chosen as the four meteorological variables of the model. Soil temperature gives more robust information than air temperature alone, wind impacts heating losses, and atmospheric pressure gives quantitative estimates of the outside weather.

To validate or invalidate this choice, a GAM was fitted and nullity tests were done as shown later.

4.3 The CO₂ emissions' role

Even though a combination of the previously discussed variables may yield acceptable results, a notable component seems to be lacking, as they do not represent the global prodigality or frugality tendencies. In order to capture this effect, the annual CO₂ emissions in France were added. According to RTE ³, although electrical production is mostly low-carbon and is responsible for around 5% of the total CO₂ emissions, we suspected it could still be a good indicator of industrial activity. Low emissions would therefore be linked with lower load values. To test this theory, a GAM was fitted with the previous variables on the France load. The residuals $y - \hat{y}$ were then used in a linear regression with emissions as the explanatory variable.

³<https://analysesetdonnees.rte-france.com/en/electricity-review-keyfindings>

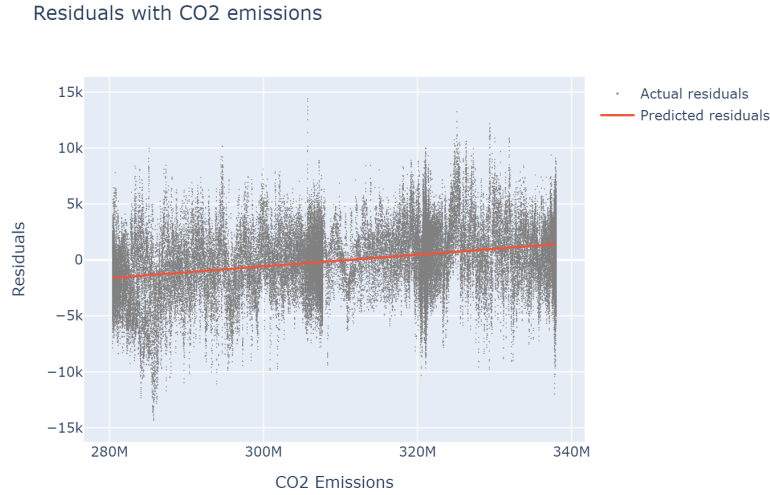


Figure 6: Linear regression on GAM residuals

A clear trend arises as predicted, with nearly a 4000MW difference between the highest and lowest predicted residuals. This is a clear sign CO₂ emissions are highly correlated with electricity consumption in France.

4.4 Statistical significativity validation

To validate or invalidate the choice of the previous explanatory variables, a GAM was once again fitted using the combination of all discussed features with averaged values to predict the electric load in France, with the exception that Fourier features were not used as they are not suited with how a GAM works. The following summary is obtained; both nullity tests and degrees of freedom reflect all variables are significant (see table 1).

5 Model, loss function, validation

5.1 Model

To better encompass economic flows and interactions that might occur between different regions of France, and in an effort to reduce training time, the proposed model predicts all $m = 25$ modalities at once and inputs all covariables.

An LSTM needs an input in the form of a sequence of predetermined length T . Of course, when using this model to predict new data, a *warmup* set is then concatenated before the start of the test set. Let $S \in \mathbb{R}^{T \times K}$ a sequence of length T using the concatenation of all the K features discussed beforehand, the prediction is then given by $\text{Linear}(\text{LSTM}(S)_T) \in \mathbb{R}^m$, where LSTM and Linear are mappings such

that $\text{LSTM} : \mathbb{R}^{T \times K} \rightarrow \mathbb{R}^{T \times h}$ and $\text{Linear} : \mathbb{R}^h \rightarrow \mathbb{R}^m$, h being the hidden dimension of the LSTM, which discards the memory parameters.

5.2 Loss function

Furthermore, to remedy the missing values issue in the cities' electrical consumption, as shown before, without requiring the imputation of this data, a custom loss function has been defined as a class reflecting the challenge's actual loss function.

Let Y_{ij} designate the load value at the instant i for the location j , such that the matrix $Y = (Y_{ij})_{ij} \in \mathbb{R}^{n \times m}$ is in direct correspondence with the target dataframe. Furthermore, allow $Y_{ij} \in \mathbb{R}^+ \cup \{-1\}$, a value of -1 hereby representing a missing value (this notation is purely for mathematical formalization of the problem). Let us denote $Y^{(B)} \in \mathbb{R}^{B \times m}$ any sampling of B rows of Y , corresponding to a given batch. Then, for a prediction $\hat{Y} \in \mathbb{R}^{B \times m}$, the loss function is defined as :

$$B_j = \sum_{i=1}^B 1_{Y_{ij} \neq -1},$$

$$l(\hat{Y}, Y^{(B)}) = \sum_{j=1}^m \sqrt{\frac{1}{B_j} \sum_{i=1}^{B_j} 1_{Y_{ij} \neq -1} (\hat{Y}_{ij} - Y_{ij})^2}.$$

If $B_j = 0$ (which is very unlikely given the large size of the dataset), we set the value of the inner summation to 0 by convention.

Since targets $\tilde{Y}$ are also scaled, say by a factor of $\frac{1}{\sigma_j}$ each, the loss is simply rewritten as:

$$l(\hat{Y}, \tilde{Y}^{(B)}) = \sum_{j=1}^m \sigma_j \sqrt{\frac{1}{B_j} \sum_{i=1}^{B_j} 1_{Y_{ij} \neq -1} (\hat{Y}_{ij} - \tilde{Y}_{ij})^2}.$$

5.3 Hyperparameters

For the LSTM, we settled on a sequence length of $T = 48$, which corresponds to a full day of recordings. Additionally, L^2 Ridge-like penalization with $\lambda = 0.01$ was employed to prevent overfitting in conjunction with the Adam optimizer [7]. A learning rate of $\eta = 0.001$ was used and not optimized. For the LSTM, two hidden layers were set with 256 neurons each, without any dropout. The final fully connected layer is an affine transformation from $\mathbb{R}^{256} \rightarrow \mathbb{R}^{25}$. We trained the neural network for 10 epochs using a batch size of $B = 128$.

This configuration yields a model with 1 049 881 parameters, which is less than the input degrees of freedom of $25 \times 85\,571 = 2\,139\,275$.

5.4 Validation

The architecture and the hyperparameters were validated using the last 15% of the training dataset at our disposal, without shuffling, in order not to interpolate training data, therefore yielding an unbiased estimate for the error.

6 Results

The training of our first model (that doesn't consider the validation set) yields promising results. We observe that our training and validation losses decline rapidly over the first iterations of the training loop (see figure A2). We obtain a final validation loss of **4910.3335** with the total RMSE loss, and a MAPE loss on France of **2.35%**, which is the best we managed to obtain. Note this MAPE performance is quite remarkable since we do not even optimize this criterion. With the MLP model, we obtained a slightly higher validation loss. Finally, we train our definitive model on all the available data. We obtain a training loss of **1262.5434**, and a public score of **6068.28** in the Codabench competition ⁴. We also illustrate the difference between predictions and the true values at different time scales on the validation set.

Predictions on the validation set

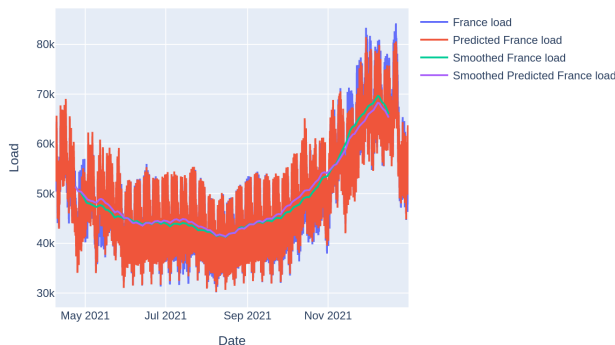


Figure 7: True vs predicted France electricity load over a year

Predictions on the first week of the validation set

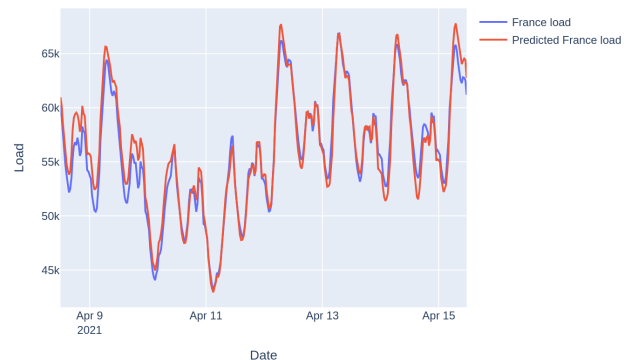


Figure 8: True vs predicted France electricity load over a week

The errors have also been plotted to check for their normality, and a QQ Plot of the residuals was drawn. As one can see, although some variations are noted, residuals calculated on the validation set are very close to being normal. The residuals were calculated based on the total France load (see figures A3 and A4).

⁴<https://www.codabench.org/competitions/5206>

7 Conclusion and discussion

We presented techniques and results for year-long predictions of electrical load, using French electricity data. We added date-encoding variables, weather values, and carbon emissions. We also modified the loss function to make our training work better, even with missing data. By using an LSTM instead of an MLP, we aimed at enhancing performance by making it easier for the model to deduce time-related dependencies that may exist in our data. All of this earned us first place in the public Codabench competition.

In practice, electrical companies are faced daily with load forecasting problems. However, it's important to note that the problems in their case often have much shorter time horizons. By focusing on predictions over a whole calendar year, we partially lose a whole set of tools that can be useful in these kinds of problems (e.g. auto-regressive models and online learning). Furthermore, assuming that we have perfectly accurate forecasts on carbon emissions and weather variables is also an unrealistic assumption, especially for weather variables. Weather forecasts don't usually go beyond 10 days, and their accuracy decreases with the number of days ahead they're trying to predict. The model's performance could potentially be improved by working with regularly updated data. It is also true that the choice of variables is still somewhat arbitrary and could be refined. The weather data that we were provided with contained many missing values. Even though we justified the interest of the variables we ended up choosing, there are many that we never tested and that we couldn't have tested, given the deadline. Finally, future work could also help us optimize the model architecture and hyperparameters (sequence length, batch size, number of neurons and layers), by using AutoML techniques [8]. In this case, hyperparameters were manually fixed, after some slight experimentation.

8 Appendix

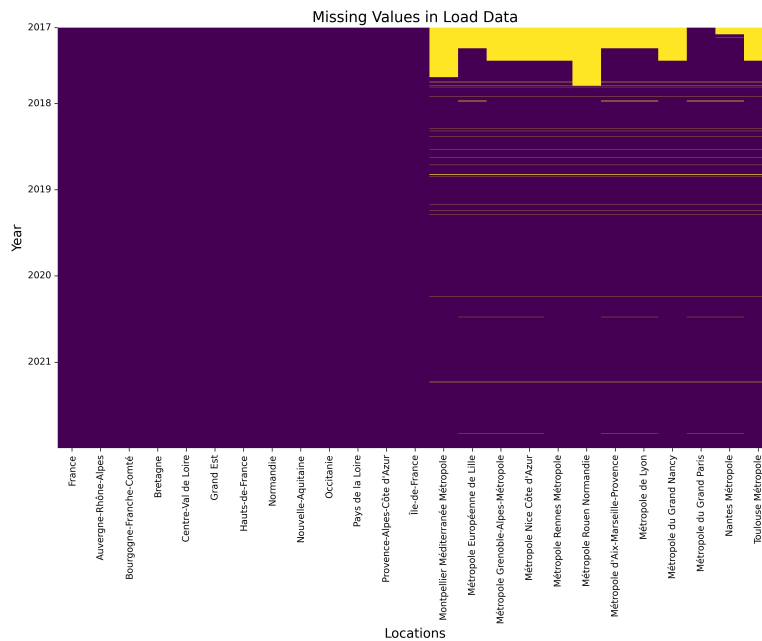


Figure A1: The missing values from the load dataset

Model Summary	Feature Function Statistics					
Distribution: NormalDist	Feature Function	Lambda	Rank	EDoF	P > x	Sig. Code
Link Function: IdentityLink	te(0, 1)	[0.6, 0.6]	100	45.3	1.11e-16	***
Number of Samples: 85571	s(2)	[0.6]	20	15.7	1.11e-16	***
Effective DoF: 142.3072	s(3)	[0.6]	20	16.3	1.11e-16	***
Log Likelihood: -1436449.0606	s(4)	[0.6]	20	16.3	1.11e-16	***
AIC: 2873184.7356	s(5)	[0.6]	20	18.3	1.11e-16	***
AICc: 2873185.2198	s(6)	[0.6]	20	18.6	1.11e-16	***
GCV: 7808261.7735	f(7)	[0.6]	12	10.9	1.11e-16	***
Scale: 7784891.5539	l(8)	[0.6]	1	1.0	1.11e-16	***
Pseudo R-Squared: 0.9414	intercept	-	1	0.0	1.11e-16	***

Table 1: LinearGAM Summary and Feature Function Statistics

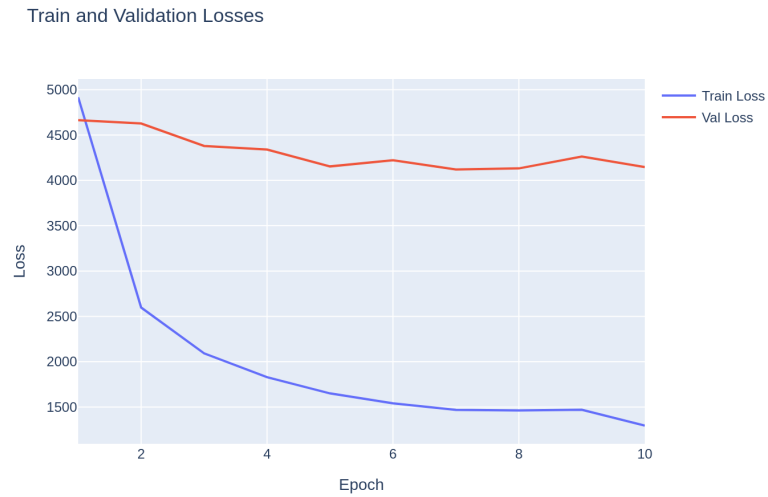


Figure A2: Train and validation loss

Histogram of errors

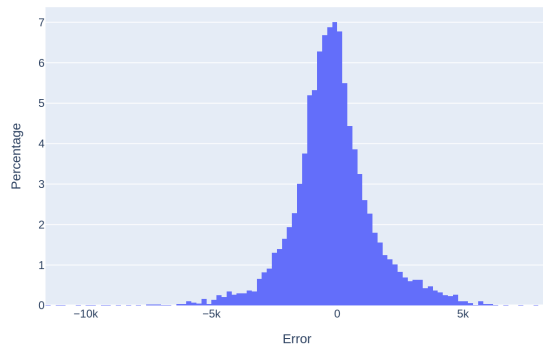


Figure A3: Histogram of the residuals on the validation set

QQ Plot of standardized residuals

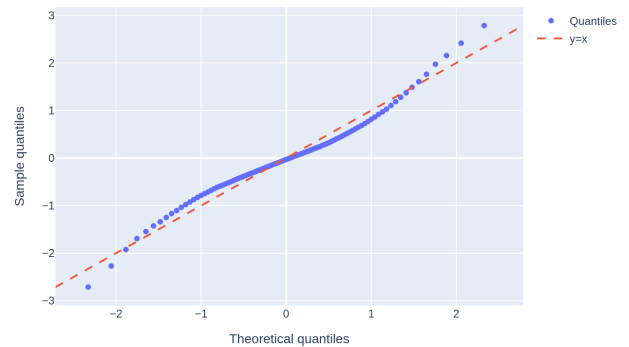


Figure A4: QQ Plot of the standardized residuals

References

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, pp. 1735–1780, 11 1997.
- [2] E. Badfar, E. Jalaieian, and A. Nikoofard, “Design a robust sliding mode controller based on the state and parameter estimation for the nonlinear epidemiological model of covid-19,” *Nonlinear Dynamics*, vol. 109, 07 2022.
- [3] D. Park, M. El-Sharkawi, R. Marks, L. Atlas, and M. Damborg, “Electric load forecasting using an artificial neural network,” *IEEE Transactions on Power Systems*, vol. 6, no. 2, pp. 442–449, 1991.
- [4] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?,” 2022.
- [5] D. L. Marino, K. Amarasinghe, and M. Manic, “Building energy load forecasting using deep neural networks,” *CoRR*, vol. abs/1610.09460, 2016.
- [6] M. Abumohsen, A. Owda, and M. Owda, “Electrical load forecasting using lstm, gru, and rnn algorithms,” *Energies*, vol. 16, p. 2283, 02 2023.
- [7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [8] J. Keisler, E.-G. Talbi, S. Claudel, and G. Cabriel, “An algorithmic framework for the optimization of deep neural networks architectures and hyperparameters,” *Journal of Machine Learning Research*, vol. 25, no. 201, pp. 1–33, 2024.